

MYLIFE HEALTHCARE PLATFORM

Milestone 3: REST API Design and Implementation

Group 27 - Team Members	
T.M.D.P.M De Alwis	EC/2022/012
K.G.H.D Bandara	EC/2022/037
B.T. Bhanuka	EC/2022/016
A.T.I Karunathilake	EC/2022/055

Table of Contents

1. APPLICATION OVERVIEW	3
2. MICROSERVICES ARCHITECTURE	4
3. REST API ENDPOINTS.....	5
3.1 AUTH SERVICE (PORT 8001 PREFIX: /AUTH)	5
3.2 MEDICAL RECORDS SERVICE (PORT 8002 PREFIX: /RECORDS).....	5
3.3 APPOINTMENT SUB-SERVICE (PORT 8002 PREFIX: /APPOINTMENTS)	6
3.4 EMERGENCY PROFILE SUB-SERVICE (PORT 8002 PREFIX: /EMERGENCY)	7
3.5 FAMILY & PROFILE SERVICE (PORT 8003 PREFIX: /FAMILY)	7
3.6 AI PROCESSING SERVICE (PORT 8004 PREFIX: /AI)	8
3.7 NOTIFICATION SERVICE (PORT 8005 PREFIX: /NOTIFY)	8
4. KEY DATA MODELS (DATABASE TABLES)	9
5. SECURITY & AUTHENTICATION	11
6. DEPLOYMENT ARCHITECTURE.....	11
7. SOURCE CODE REPOSITORY	12
8. VIDEO DEMONSTRATION.....	12
9. REST API CODE/SCREENSHOTS	13

1. Application Overview

MyLife is a modern personal healthcare management platform built on a microservices architecture. It allows patients to manage medical records, book appointments with doctors, set up emergency profiles, link family members, scan medical documents with AI, and receive real-time notifications. The platform is deployed on AWS EC2 behind an NGINX API Gateway and the React/TypeScript frontend is hosted on Vercel.

Technology Stack

- **Backend Framework:** FastAPI (Python 3.11) for all 5 microservices
- **Database & Auth:** Supabase (PostgreSQL) — hosted, managed Postgres with PostgREST
- **File Storage:** AWS S3 (boto3) for medical document uploads via presigned URLs
- **AI Vision:** Google Gemini 2.5 Flash — multimodal vision model for document OCR and medical extraction
- **Containerisation:** Docker + Docker Compose — all services in isolated containers
- **API Gateway:** NGINX (reverse proxy, SSL termination, HTTPS via Let's Encrypt)
- **Frontend:** React 18 + TypeScript + Vite, deployed on Vercel
- **CI/CD:** GitHub Actions — automated deploy to AWS EC2 on push to main
- **Notifications:** Firebase Cloud Messaging (push) + Supabase Edge Functions (email)
- **Authentication:** JWT (HS256) — access tokens (30 min) + refresh tokens (7 days)

2. Microservices Architecture

The platform is composed of 5 independent RESTful microservices, each running in its own Docker container and communicating via internal Docker network. All external requests are routed through the NGINX gateway.

Service	Container	Port	Prefix	Responsibility
Auth Service	mylife-auth	8001	/auth	User registration, login, JWT issuance, doctor listing
Medical Records Service	mylife-medical	8002	/records, /appointments , /emergency	CRUD for records, appointments, and emergency profiles
Family & Profile Service	mylife-family	8003	/family	Link/unlink family members, view their records
AI Processing Service	mylife-ai	8004	/ai	Gemini Vision document scanning, extraction storage
Notification Service	mylife-notification	8005	/notify	Email & push notifications, reminder scheduling

3. REST API Endpoints

Below are all implemented RESTful endpoints grouped by microservice. All authenticated endpoints require the header: Authorization: Bearer <access_token>

3.1 Auth Service (Port 8001 | Prefix: /auth)

Endpoint	HTTP Method	Description	Technology	Auth Required
POST /auth/register	POST	Register a new user (patient or doctor). Returns JWT tokens.	FastAPI, Supabase	No
POST /auth/login	POST	Authenticate user with email/password. Returns access + refresh tokens.	FastAPI, Supabase	No
POST /auth/refresh	POST	Issue a new access token using a valid refresh token.	FastAPI, JWT	Refresh token
POST /auth/logout	POST	Invalidate session (client discards JWT). Stateless logout.	FastAPI, JWT	Yes
GET /auth/me	GET	Retrieve the currently authenticated user's profile.	FastAPI, Supabase	Yes
PUT /auth/me	PUT	Update the current user's full name.	FastAPI, Supabase	Yes
GET /auth/doctors	GET	List all registered doctors (for appointment booking by patients).	FastAPI, Supabase	Yes

3.2 Medical Records Service (Port 8002 | Prefix: /records)

Endpoint	HTTP Method	Description	Technology	Auth Required
GET /records/	GET	List all medical records for the authenticated user.	FastAPI, Supabase, boto3	Yes
POST /records/	POST	Create a new medical record entry.	FastAPI, Supabase	Yes
GET /records/{record_id}	GET	Retrieve a specific record by ID (ownership enforced).	FastAPI, Supabase	Yes
PUT /records/{record_id}	PUT	Update an existing medical record.	FastAPI, Supabase	Yes
DELETE /records/{record_id}	DELETE	Delete a record (ownership enforced).	FastAPI, Supabase	Yes

GET /records/presign-upload	GET	Generate an AWS S3 presigned PUT URL for direct browser upload.	FastAPI, AWS S3, boto3	Yes
POST /records/confirm-upload	POST	Confirm a successful S3 upload and trigger AI extraction.	FastAPI, httpx, AWS S3	Yes
POST /records/upload	POST	Legacy: stream a file through backend to S3 (fallback).	FastAPI, AWS S3, boto3	Yes
GET /records/family/{id}	GET	View medical records of a linked family member.	FastAPI, Supabase	Yes
POST /records/share-qr	POST	Generate a QR share token for a specific record.	FastAPI, Supabase	Yes

3.3 Appointment Sub-Service (Port 8002 | Prefix: /appointments)

Endpoint	HTTP Method	Description	Technology	Auth Required
POST /appointments /	POST	Patient books an appointment with a doctor.	FastAPI, Supabase	Yes (patient only)
GET /appointments /mine	GET	Retrieve all appointments for the logged-in patient.	FastAPI, Supabase	Yes
GET /appointments /doctor	GET	Retrieve all appointments assigned to a logged-in doctor.	FastAPI, Supabase	Yes (doctor only)
PUT /appointments /{appointment_id}	PUT	Update appointment status (confirmed/completed/cancelled).	FastAPI, Supabase	Yes
DELETE /appointments /{appointment_id}	DELETE	Cancel and delete an appointment.	FastAPI, Supabase	Yes

3.4 Emergency Profile Sub-Service (Port 8002 | Prefix: /emergency)

Endpoint	HTTP Method	Description	Technology	Auth Required
GET /emergency/profile/{user_id}	GET	Public endpoint — returns emergency profile for responders (QR scan).	FastAPI, Supabase	No (public)
POST /emergency/profile	POST	Create the current user's emergency profile (blood type, allergies, etc.).	FastAPI, Supabase	Yes
PUT /emergency/profile	PUT	Update the current user's emergency profile.	FastAPI, Supabase	Yes
POST /emergency/profile/upsert	POST	Create or update emergency profile in a single call (upsert).	FastAPI, Supabase	Yes

3.5 Family & Profile Service (Port 8003 | Prefix: /family)

Endpoint	HTTP Method	Description	Technology	Auth Required
POST /family/link	POST	Link another registered user as a family member with a relationship label.	FastAPI, Supabase	Yes
GET /family/members	GET	List all linked family members with their user details.	FastAPI, Supabase	Yes
DELETE /family/unlink/{linked_user_id}	DELETE	Remove a family member link.	FastAPI, Supabase	Yes

3.6 AI Processing Service (Port 8004 | Prefix: /ai)

Endpoint	HTTP Method	Description	Technology	Auth Required
POST /ai/process	POST	Accept a file URL, download it, send to Gemini Vision, store structured results.	FastAPI, Gemini API, Supabase	No (internal)
POST /ai/process-upload	POST	Accept direct file upload (multipart), analyse with Gemini Vision.	FastAPI, Gemini API, Supabase	No (internal)
GET /ai/results/{doc_id}	GET	Retrieve AI extraction results for a specific document ID.	FastAPI, Supabase	No
GET /ai/summary	GET	Return all extraction results for a given user_id (scan history).	FastAPI, Supabase	No

3.7 Notification Service (Port 8005 | Prefix: /notify)

Endpoint	HTTP Method	Description	Technology	Auth Required
POST /notify/email	POST	Send a transactional email for a given event (record_created, ai_extraction_complete, etc.).	FastAPI, Supabase Edge Functions	No (internal)
POST /notify/push	POST	Send a Firebase Cloud Messaging push notification to a user's device.	FastAPI, Firebase Admin SDK	No (internal)
POST /notify/reminder	POST	Schedule a future reminder (appointment or medication) for a user.	FastAPI, Supabase	No (internal)

4. Key Data Models (Database Tables)

The following Supabase (PostgreSQL) tables are used across all microservices:

users (auth schema)

Columns
id (UUID PK)
email
full_name
password_hash
role (patient doctor)
gender
created_at

appointments (public schema)

Columns
id (UUID PK)
patient_id (FK→users)
doctor_id (FK→users)
scheduled_at
reason
status (pending confirmed completed cancelled)
notes

medical_records (public schema)

Columns
id (UUID PK)
user_id (FK→users)
title
record_type
description
doctor_name
visit_date
diagnosis
file_url
created_at

emergency_profiles (public schema)

Columns
id (UUID PK)
user_id (FK→users)
blood_type
allergies []
chronic_conditions []
emergency_contact_name
emergency_contact_phone
current_medications []

linked_accounts (public schema)

Columns
id (UUID PK)
owner_id (FK→users)
linked_user_id (FK→users)
relationship
created_at

uploaded_documents (public schema)

Columns
id (UUID PK)
user_id (FK→users)
file_url
status (processing completed failed)
created_at

extracted_reports (public schema)

Columns
id (UUID PK)
document_id (FK→uploaded_documents)
user_id (FK→users)
extracted_data (JSONB)
confidence_score (float)
created_at

notification_logs (public schema)

Columns
id (UUID PK)
user_id (FK→users)
channel (email push)
event
status
created_at

shared_records (public schema)

Columns
id (UUID PK)
record_id (FK→medical_records)
token (UUID)
expires_at
created_by (FK→users)

5. Security & Authentication

- **JWT Authentication:** All protected endpoints validate a Bearer token in the Authorization header. Tokens are HS256 JWTs signed with a server-side secret.
- **Access Tokens:** Short-lived (30 minutes). Contain `user_id`, email, and role claims.
- **Refresh Tokens:** Long-lived (7 days). Used exclusively at POST `/auth/refresh` to issue new access tokens.
- **Password Hashing:** bcrypt via passlib — passwords are never stored in plain text.
- **Ownership Enforcement:** All record/appointment queries include `.eq('user_id', current_user['sub'])` to prevent cross-user data access.
- **Role-Based Access:** The 'role' claim (patient/doctor) controls which endpoints are accessible. E.g. only patients can book appointments; only doctors can view their queue.
- **S3 Presigned URLs:** Medical file uploads use short-lived (15-minute) S3 presigned PUT URLs so files never traverse the backend server.
- **Public Emergency Endpoint:** GET `/emergency/profile/{user_id}` is intentionally public to allow emergency responders to access critical info via QR code without authentication.

6. Deployment Architecture

The backend is fully containerised using Docker and deployed on an AWS EC2 instance. NGINX acts as a reverse proxy and SSL termination point using Let's Encrypt certificates. The frontend is separately deployed on Vercel with environment variables pointing to the EC2 API.

- **Production URL:** <https://myhealth.jo3.org> (EC2 + NGINX + Let's Encrypt SSL)
- **Frontend URL:** <https://mylife.vercel.app> (Vercel, React/TypeScript)
- **CI/CD:** GitHub Actions (`.github/workflows/deploy.yml`) — SSH deploy on push to main
- **Container Orchestration:** Docker Compose v3.9 — all 5 services + NGINX in one compose file
- **Health Checks:** Each service exposes GET `/health` — Docker monitors and restarts on failure
- **Networking:** All containers on mylife-network bridge — internal DNS resolution by service name

7. Source Code Repository

The complete source code for both the backend (microservices) and frontend (React app) is available on GitHub. The repository includes all service implementations, Docker configurations, NGINX gateway config, database schema, CI/CD workflows, and documentation.

- **Backend Repo:** <https://github.com/Zeero-G/MyLife-HealthCare-Backend>
- **Frontend Repo:** <https://github.com/Zeero-G/MyLife-HealthCare-Frontend>
- **Structure:** Services: auth-service, medical-records-service, family-profile-service, ai-processing-service, notification-service
- **Each Service:** All services include: Dockerfile, requirements.txt, .env.example, FastAPI application

8. Video demonstration

A video demonstration of the MyLife Healthcare Platform has been prepared to showcase the key functionalities implemented throughout the project. The demonstration covers user registration and authentication, medical record management, appointment booking, emergency profile access, family account linking, AI-powered medical document scanning, and notification features. It also highlights the microservices architecture, API gateway integration, cloud deployment, and end-to-end system workflow. The video provides a practical walkthrough of the platform from both the patient and healthcare provider perspectives, demonstrating the successful implementation of the designed REST APIs and the interaction between the frontend, backend microservices, database, cloud storage, and AI processing components.

Video Demonstration Link:

https://drive.google.com/drive/folders/1aicIa3I7AoFI1z6ocH9EZPD4p-1XGZpP?usp=drive_link

9. REST API code/screenshots

1. Auth Service

services/auth-service/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import auth
from app.core.config import settings

app = FastAPI(
    title="MYLIFE Auth Service",
    description="Handles all user identity, authentication, roles, and permissions.",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth.router, prefix="/auth", tags=["Auth"])

@app.get("/health")
def health_check():
    return {"service": "auth-service", "status": "ok"}
```

services/auth-service/app/routers/auth.py

```
from fastapi import APIRouter, HTTPException, status, Depends
from app.schemas.auth_schemas import (
    RegisterRequest, LoginRequest, RefreshRequest,
    PasswordResetRequest, TokenResponse, UserResponse,
)
from app.core.security import (
    hash_password, verify_password,
    create_access_token, create_refresh_token, decode_token, get_current_user,
)
from app.core.database import supabase

router = APIRouter()

@router.post("/register", response_model=TokenResponse, status_code=status.HTTP_201_CREATED)
async def register(payload: RegisterRequest):
    existing = supabase.table("users").select("id").eq("email", payload.email).execute()
    if existing.data:
        raise HTTPException(status_code=409, detail="Email already registered")

    hashed = hash_password(payload.password)
```

```

new_user = supabase.table("users").insert({
    "email": payload.email,
    "full_name": payload.full_name,
    "password_hash": hashed,
    "role": payload.role,
    "gender": payload.gender,
}).execute()

user_id = new_user.data[0]["id"]
token_data = {"sub": user_id, "email": payload.email, "role": payload.role}

return TokenResponse(
    access_token=create_access_token(token_data),
    refresh_token=create_refresh_token(token_data),
)

@router.post("/login", response_model=TokenResponse)
async def login(payload: LoginRequest):
    result = supabase.table("users").select("*").eq("email", payload.email).execute()
    if not result.data:
        raise HTTPException(status_code=401, detail="Invalid credentials")

    user = result.data[0]
    if not verify_password(payload.password, user["password_hash"]):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    token_data = {"sub": user["id"], "email": user["email"], "role": user["role"]}
    return TokenResponse(
        access_token=create_access_token(token_data),
        refresh_token=create_refresh_token(token_data),
    )

@router.post("/refresh", response_model=TokenResponse)
async def refresh(payload: RefreshRequest):
    decoded = decode_token(payload.refresh_token)
    if decoded.get("type") != "refresh":
        raise HTTPException(status_code=401, detail="Invalid token type")

    token_data = {"sub": decoded["sub"], "email": decoded["email"], "role": decoded["role"]}
    return TokenResponse(
        access_token=create_access_token(token_data),
        refresh_token=create_refresh_token(token_data),
    )

@router.post("/logout")
async def logout(current_user: dict = Depends(get_current_user)):

```

```

        return {"message": "Logged out successfully"}

@router.get("/me", response_model=UserResponse)
async def me(current_user: dict = Depends(get_current_user)):
    result = supabase.table("users").select("*").eq("id", current_user["sub"]).execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="User not found")
    u = result.data[0]
    return UserResponse(id=u["id"], email=u["email"], full_name=u["full_name"], role=u["role"],
gender=u.get("gender"))

@router.get("/doctors")
async def list_doctors(current_user: dict = Depends(get_current_user)):
    result = supabase.table("users") \
        .select("id, full_name, email, gender") \
        .eq("role", "doctor") \
        .execute()
    return result.data

from pydantic import BaseModel

class UpdateProfileRequest(BaseModel):
    full_name: str

@router.put("/me", response_model=UserResponse)
async def update_me(payload: UpdateProfileRequest, current_user: dict =
Depends(get_current_user)):
    result = supabase.table("users") \
        .update({"full_name": payload.full_name}) \
        .eq("id", current_user["sub"]) \
        .execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="User not found")
    u = result.data[0]
    return UserResponse(id=u["id"], email=u["email"], full_name=u["full_name"], role=u["role"],
gender=u.get("gender"))

```

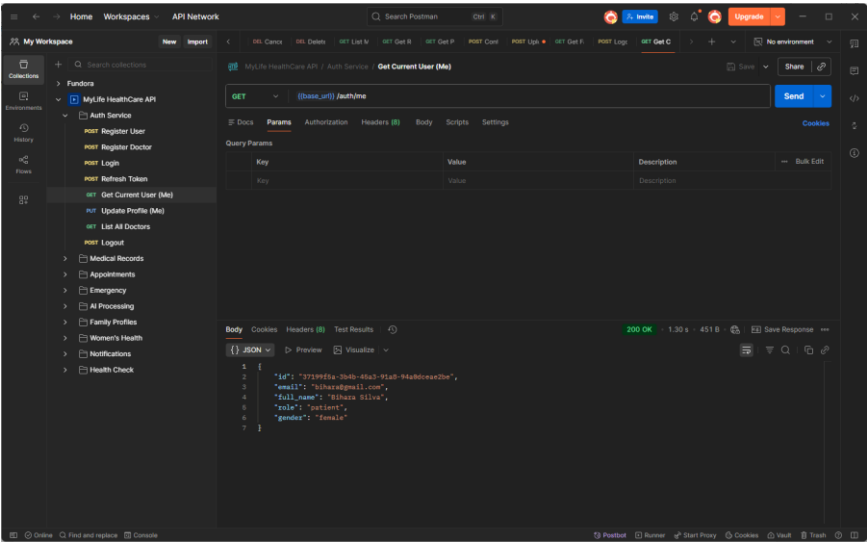
The screenshot displays the Visual Studio Code editor with a project named 'Visual Studio Workspace'. The 'EXPLORER' sidebar on the left shows the project structure, including a 'src' directory with sub-directories 'main' and 'utils'. The 'main' directory contains a 'main.ts' file, and the 'utils' directory contains a 'utils.ts' file. The 'main.ts' file is open in the editor, showing a TypeScript class named 'Main' with a 'run' method. The 'utils.ts' file is also open in the editor, showing a 'utils' namespace with a 'log' function. The 'SEARCH' sidebar on the right shows search results for the 'main.ts' file. The 'RUN AND DEBUG' sidebar on the right shows the 'Run and Debug' configuration. The 'main.ts' file is open in the editor, showing a TypeScript class named 'Main' with a 'run' method. The 'utils.ts' file is also open in the editor, showing a 'utils' namespace with a 'log' function.

The screenshot shows the AWS IAM console interface. On the left, the 'Groups' page is selected under the 'Users and Groups' section. The main content area displays the 'AWS-ReadOnlyAccess' group. The 'Permissions' tab is active, showing a table of attached policies. The 'AmazonS3ReadOnlyAccess' policy is listed as attached. Below this, the 'Users' tab is visible, showing a list of users. The 'Users' tab is currently empty, and the 'Groups' tab is selected.

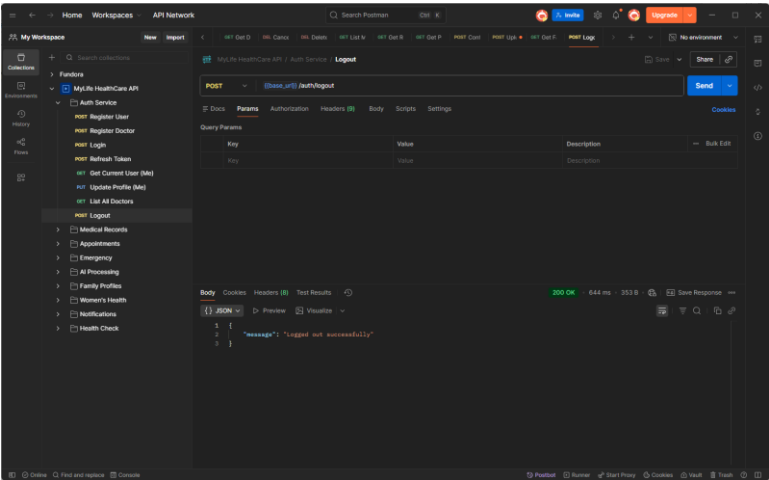
The screenshot shows the AWS IAM console interface. On the left, the 'Groups' page is selected under the 'IAMUsers' group. The 'Permissions' tab is active, displaying a table of attached policies. The 'Attach New Policy' button is highlighted. The 'AWS managed' policy list shows 'AmazonS3ReadOnlyAccess' selected.

The screenshot shows the AWS IAM console interface. On the left, the 'My Workspaces' sidebar is visible, with 'MyAppHealthCheckAPI' selected under the 'API Network' section. The main content area displays the 'POST' method configuration for the 'MyAppHealthCheckAPI' role. The 'POST' method is configured to allow the 'MyAppHealthCheckAPI' role to perform the 'POST' action on the 'POST' resource. The 'POST' method is configured to allow the 'MyAppHealthCheckAPI' role to perform the 'POST' action on the 'POST' resource. The 'POST' method is configured to allow the 'MyAppHealthCheckAPI' role to perform the 'POST' action on the 'POST' resource.

GET/auth/me



POST/auth/logout



2.) Medical Service

services/medical-records-service/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import records, emergency, appointments
from app.core.config import settings

app = FastAPI(
    title="MYLIFE Medical Records Service",
    description="Core service: manages all health records, QR sharing,
and emergency profiles.",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(records.router, prefix="/records", tags=["Records"])
app.include_router(emergency.router, prefix="/emergency",
tags=["Emergency"])
app.include_router(appointments.router, prefix="/appointments",
tags=["Appointments"])

@app.get("/health")
def health_check():
    return {"service": "medical-records-service", "status": "ok"}
```

services/medical-records-service/app/routers/records.py

```
import uuid
from datetime import datetime, timedelta
from fastapi import APIRouter, HTTPException, Depends, UploadFile, File,
status, Query
import httpx
import boto3
from botocore.exceptions import ClientError, NoCredentialsError

from app.core.config import settings
from app.core.security import get_current_user
from app.core.database import supabase
from app.schemas.records_schemas import (
```

```

        CreateRecordRequest, UpdateRecordRequest, ShareQRRequest,
        RecordResponse, ShareQRResponse,
    )

    router = APIRouter()

    def _s3_client():
        if not settings.AWS_ACCESS_KEY_ID or not settings.S3_BUCKET:
            raise HTTPException(
                status_code=503,
                detail="S3 is not configured on the server. Set
AWS_ACCESS_KEY_ID and S3_BUCKET."
            )
        return boto3.client(
            "s3",
            region_name=settings.AWS_REGION,
            aws_access_key_id=settings.AWS_ACCESS_KEY_ID,
            aws_secret_access_key=settings.AWS_SECRET_ACCESS_KEY,
        )

    def _s3_public_url(key: str) -> str:
        return
f"https://{settings.S3_BUCKET}.s3.{settings.AWS_REGION}.amazonaws.com/{ke
y}"

    @router.get("/", response_model=list[RecordResponse])
    async def list_records(current_user: dict = Depends(get_current_user)):
        result = supabase.table("medical_records") \
            .select("*") \
            .eq("user_id", current_user["sub"]) \
            .order("created_at", desc=True) \
            .execute()
        return result.data

    @router.post("/share-qr", response_model=ShareQRResponse)
    async def share_qr(payload: ShareQRRequest, current_user: dict =
Depends(get_current_user)):
        record_check = supabase.table("medical_records") \
            .select("id") \
            .eq("id", payload.record_id) \
            .eq("user_id", current_user["sub"]) \
            .execute()
        if not record_check.data:
            raise HTTPException(status_code=404, detail="Record not found")

```

```

    token = str(uuid.uuid4())
    expires_at = datetime.utcnow() +
timedelta(hours=payload.expires_hours)

    supabase.table("shared_records").insert({
        "record_id": payload.record_id,
        "token": token,
        "expires_at": expires_at.isoformat(),
        "created_by": current_user["sub"],
    }).execute()

    share_url = f"https://mylife.vercel.app/shared/{token}"
    return ShareQRResponse(qr_token=token, share_url=share_url,
expires_at=expires_at.isoformat())

@router.get("/presign-upload")
async def presign_upload(
    filename: str = Query(..., description="Original filename e.g.
report.pdf"),
    content_type: str = Query("application/octet-stream",
description="MIME type of the file"),
    current_user: dict = Depends(get_current_user),
):
    s3 = _s3_client()
    key = f"{current_user['sub']}/{uuid.uuid4()}_{filename}"
    try:
        presigned_url = s3.generate_presigned_url(
            "put_object",
            Params={
                "Bucket": settings.S3_BUCKET,
                "Key": key,
                "ContentType": content_type,
            },
            ExpiresIn=settings.S3_PRESIGN_EXPIRY,
        )
    except (NoCredentialsError, ClientError) as e:
        raise HTTPException(status_code=500, detail=f"Could not generate
presigned URL: {e}")

    return {
        "presigned_url": presigned_url,
        "public_url": _s3_public_url(key),
        "key": key,
        "expires_in": settings.S3_PRESIGN_EXPIRY,
    }

@router.post("/confirm-upload")

```

```

async def confirm_upload(
    payload: dict,
    current_user: dict = Depends(get_current_user),
):
    file_url: str = payload.get("file_url", "")
    if not file_url:
        raise HTTPException(status_code=422, detail="file_url is
required")

    try:
        async with httpx.AsyncClient() as client:
            await client.post(
                f"{settings.AI_SERVICE_URL}/ai/process",
                json={"user_id": current_user["sub"], "file_url":
file_url},
                timeout=10,
            )
    except Exception:
        pass

    return {
        "file_url": file_url,
        "message": "Upload confirmed. AI extraction queued.",
    }

@router.post("/upload")
async def upload_document(file: UploadFile = File(...), current_user:
dict = Depends(get_current_user)):
    s3 = _s3_client()
    file_bytes = await file.read()
    key = f"{current_user['sub']}/{uuid.uuid4()}_{file.filename}"

    try:
        s3.put_object(
            Bucket=settings.S3_BUCKET,
            Key=key,
            Body=file_bytes,
            ContentType=file.content_type or "application/octet-stream",
        )
    except (NoCredentialsError, ClientError) as e:
        raise HTTPException(status_code=500, detail=f"S3 upload failed:
{e}")

    public_url = _s3_public_url(key)

    try:
        async with httpx.AsyncClient() as client:
            await client.post(

```

```

        f"{settings.AI_SERVICE_URL}/ai/process",
        json={"user_id": current_user["sub"], "file_url":
public_url},
        timeout=10,
    )
    except Exception:
        pass

    return {"file_url": public_url, "message": "File uploaded to S3. AI
extraction queued."}

@router.get("/family/{patient_id}", response_model=list[RecordResponse])
async def list_family_records(patient_id: str, current_user: dict =
Depends(get_current_user)):
    link_check = supabase.table("linked_accounts") \
        .select("*") \
        .eq("owner_id", current_user["sub"]) \
        .eq("linked_user_id", patient_id) \
        .execute()

    if not link_check.data:
        raise HTTPException(status_code=403, detail="Not authorized to
view these records")

    result = supabase.table("medical_records") \
        .select("*") \
        .eq("user_id", patient_id) \
        .order("created_at", desc=True) \
        .execute()
    return result.data

@router.post("/", response_model=RecordResponse, status_code=201)
async def create_record(payload: CreateRecordRequest, current_user: dict
= Depends(get_current_user)):
    new_record = {
        "user_id": current_user["sub"],
        "title": payload.title,
        "record_type": payload.record_type,
        "description": payload.description,
        "doctor_name": payload.doctor_name,
        "visit_date": str(payload.visit_date) if payload.visit_date else
None,
        "diagnosis": payload.diagnosis,
        "file_url": payload.file_url,
    }
    result =
supabase.table("medical_records").insert(new_record).execute()

```

```

try:
    async with httpx.AsyncClient() as client:
        await client.post(
            f"{settings.NOTIFICATION_SERVICE_URL}/notify/email",
            json={"user_id": current_user["sub"], "event":
"record_created", "record_title": payload.title},
            timeout=5,
        )
except Exception:
    pass

return result.data[0]

@router.get("/{record_id}", response_model=RecordResponse)
async def get_record(record_id: str, current_user: dict =
Depends(get_current_user)):
    result = supabase.table("medical_records") \
        .select("*") \
        .eq("id", record_id) \
        .eq("user_id", current_user["sub"]) \
        .execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Record not found")
    return result.data[0]

@router.put("/{record_id}", response_model=RecordResponse)
async def update_record(record_id: str, payload: UpdateRecordRequest,
current_user: dict = Depends(get_current_user)):
    update_data = {k: v for k, v in payload.model_dump().items() if v is
not None}
    result = supabase.table("medical_records") \
        .update(update_data) \
        .eq("id", record_id) \
        .eq("user_id", current_user["sub"]) \
        .execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Record not found")
    return result.data[0]

@router.delete("/{record_id}", status_code=204)
async def delete_record(record_id: str, current_user: dict =
Depends(get_current_user)):
    check = supabase.table("medical_records") \
        .select("id") \
        .eq("id", record_id) \

```

```

        .eq("user_id", current_user["sub"]) \
        .execute()
    if not check.data:
        raise HTTPException(status_code=404, detail="Record not found or
not owned by you")

    supabase.table("medical_records") \
        .delete() \
        .eq("id", record_id) \
        .eq("user_id", current_user["sub"]) \
        .execute()

```

services/medical-records-service/app/routers/appointments.py

```

from fastapi import APIRouter, HTTPException, Depends, status
from pydantic import BaseModel
from typing import Optional
from app.core.security import get_current_user
from app.core.database import supabase_public, supabase_auth

router = APIRouter()

class AppointmentCreate(BaseModel):
    doctor_id: str
    scheduled_at: str
    reason: Optional[str] = None

class AppointmentUpdate(BaseModel):
    status: Optional[str] = None
    notes: Optional[str] = None
    scheduled_at: Optional[str] = None

@router.post("/", status_code=201)
async def book_appointment(
    payload: AppointmentCreate,
    current_user: dict = Depends(get_current_user),
):
    if current_user.get("role") != "patient":
        raise HTTPException(status_code=403, detail="Only patients can
book appointments")

    doctor = supabase_auth.table("users") \
        .select("id, full_name, email") \
        .eq("id", payload.doctor_id) \
        .eq("role", "doctor") \

```



```

        .execute()
    if not doctor.data:
        raise HTTPException(status_code=404, detail="Doctor not found")

    result = supabase_public.table("appointments").insert({
        "patient_id": current_user["sub"],
        "doctor_id": payload.doctor_id,
        "scheduled_at": payload.scheduled_at,
        "reason": payload.reason,
        "status": "pending",
    }).execute()

    return {
        **result.data[0],
        "doctor_name": doctor.data[0]["full_name"],
    }

@router.get("/mine")
async def get_my_appointments(current_user: dict =
Depends(get_current_user)):
    result = supabase_public.table("appointments") \
        .select("*") \
        .eq("patient_id", current_user["sub"]) \
        .order("scheduled_at", desc=True) \
        .execute()

    enriched = []
    for appt in result.data:
        doctor = supabase_auth.table("users") \
            .select("full_name, email") \
            .eq("id", appt["doctor_id"]) \
            .execute()
        enriched.append({
            **appt,
            "doctor_name": doctor.data[0]["full_name"] if doctor.data
        else "Unknown",
            "doctor_email": doctor.data[0]["email"] if doctor.data else
        "",
        })
    return enriched

@router.get("/doctor")
async def get_doctor_appointments(current_user: dict =
Depends(get_current_user)):
    if current_user.get("role") != "doctor":
        raise HTTPException(status_code=403, detail="Only doctors can
access this endpoint")

```

```

result = supabase_public.table("appointments") \
    .select("*") \
    .eq("doctor_id", current_user["sub"]) \
    .order("scheduled_at", desc=True) \
    .execute()

enriched = []
for appt in result.data:
    patient = supabase_auth.table("users") \
        .select("full_name, email, gender") \
        .eq("id", appt["patient_id"]) \
        .execute()
    enriched.append({
        **appt,
        "patient_name": patient.data[0]["full_name"] if patient.data
    else "Unknown",
        "patient_email": patient.data[0]["email"] if patient.data
    else "",
        "patient_gender": patient.data[0].get("gender") if
    patient.data else None,
    })
    return enriched

@router.put("/{appointment_id}")
async def update_appointment(
    appointment_id: str,
    payload: AppointmentUpdate,
    current_user: dict = Depends(get_current_user),
):
    appt = supabase_public.table("appointments") \
        .select("*") \
        .eq("id", appointment_id) \
        .execute()
    if not appt.data:
        raise HTTPException(status_code=404, detail="Appointment not
    found")

    a = appt.data[0]
    is_doctor = current_user.get("role") == "doctor" and a["doctor_id"]
    == current_user["sub"]
    is_patient = a["patient_id"] == current_user["sub"]

    if not is_doctor and not is_patient:
        raise HTTPException(status_code=403, detail="Not authorized to
    modify this appointment")

```

```

        update_data = {k: v for k, v in payload.model_dump().items() if v is
not None}
        result = supabase_public.table("appointments") \
            .update(update_data) \
            .eq("id", appointment_id) \
            .execute()
        return result.data[0]

@router.delete("/{appointment_id}", status_code=204)
async def cancel_appointment(
    appointment_id: str,
    current_user: dict = Depends(get_current_user),
):
    appt = supabase_public.table("appointments") \
        .select("patient_id, doctor_id") \
        .eq("id", appointment_id) \
        .execute()
    if not appt.data:
        raise HTTPException(status_code=404, detail="Appointment not
found")

    a = appt.data[0]
    if a["patient_id"] != current_user["sub"] and a["doctor_id"] !=
current_user["sub"]:
        raise HTTPException(status_code=403, detail="Not authorized")

    supabase_public.table("appointments").delete().eq("id",
appointment_id).execute()

```

services/medical-records-service/app/routers/emergency.py

```

from fastapi import APIRouter, HTTPException, Depends
from pydantic import BaseModel
from typing import Optional, List
from app.core.security import get_current_user
from app.core.database import supabase
from app.schemas.records_schemas import EmergencyProfileResponse

router = APIRouter()

class EmergencyProfileUpsert(BaseModel):
    blood_type: Optional[str] = None
    allergies: Optional[List[str]] = None
    chronic_conditions: Optional[List[str]] = None
    emergency_contact_name: Optional[str] = None
    emergency_contact_phone: Optional[str] = None

```

```

        current_medications: Optional[List[str]] = None

@router.get("/profile/{user_id}",
response_model=EmergencyProfileResponse)
async def get_emergency_profile(user_id: str):
    result = supabase.table("emergency_profiles") \
        .select("*") \
        .eq("user_id", user_id) \
        .execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Emergency profile
not found")
    return result.data[0]

@router.post("/profile", response_model=EmergencyProfileResponse,
status_code=201)
async def create_emergency_profile(
    payload: EmergencyProfileUpsert,
    current_user: dict = Depends(get_current_user),
):
    existing = supabase.table("emergency_profiles") \
        .select("id") \
        .eq("user_id", current_user["sub"]) \
        .execute()
    if existing.data:
        raise HTTPException(status_code=409, detail="Emergency profile
already exists. Use PUT to update.")

    new_profile = {
        "user_id": current_user["sub"],
        "blood_type": payload.blood_type,
        "allergies": payload.allergies or [],
        "chronic_conditions": payload.chronic_conditions or [],
        "emergency_contact_name": payload.emergency_contact_name,
        "emergency_contact_phone": payload.emergency_contact_phone,
        "current_medications": payload.current_medications or [],
    }
    result =
supabase.table("emergency_profiles").insert(new_profile).execute()
    return result.data[0]

@router.put("/profile", response_model=EmergencyProfileResponse)
async def update_emergency_profile(
    payload: EmergencyProfileUpsert,
    current_user: dict = Depends(get_current_user),
):

```

```

        update_data = {k: v for k, v in payload.model_dump().items() if v is
not None}
        result = supabase.table("emergency_profiles") \
            .update(update_data) \
            .eq("user_id", current_user["sub"]) \
            .execute()
        if not result.data:
            raise HTTPException(status_code=404, detail="Emergency profile
not found. Create one first.")
        return result.data[0]

@router.post("/profile/upsert", response_model=EmergencyProfileResponse)
async def upsert_emergency_profile(
    payload: EmergencyProfileUpsert,
    current_user: dict = Depends(get_current_user),
):
    existing = supabase.table("emergency_profiles") \
        .select("id") \
        .eq("user_id", current_user["sub"]) \
        .execute()

    profile_data = {
        "blood_type": payload.blood_type,
        "allergies": payload.allergies or [],
        "chronic_conditions": payload.chronic_conditions or [],
        "emergency_contact_name": payload.emergency_contact_name,
        "emergency_contact_phone": payload.emergency_contact_phone,
        "current_medications": payload.current_medications or [],
    }

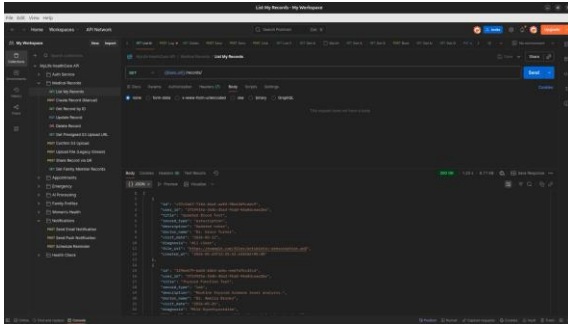
    if existing.data:
        result = supabase.table("emergency_profiles") \
            .update(profile_data) \
            .eq("user_id", current_user["sub"]) \
            .execute()
    else:
        result = supabase.table("emergency_profiles") \
            .insert({"user_id": current_user["sub"], **profile_data}) \
            .execute()

    return result.data[0]

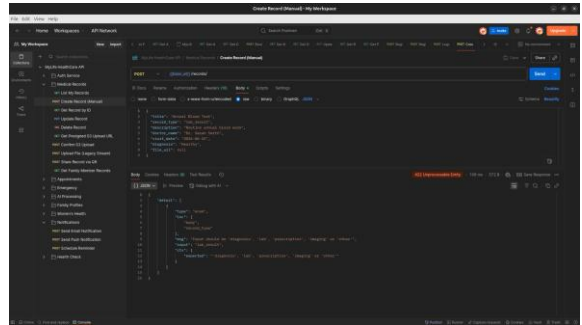
```

Medical

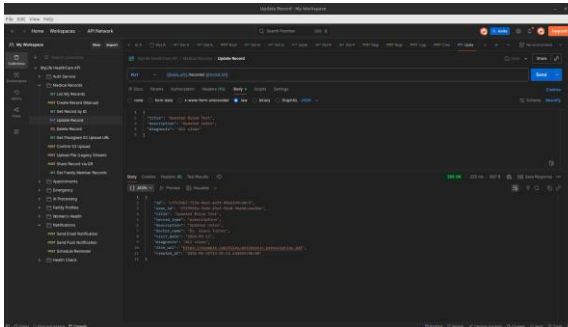
GET/records/



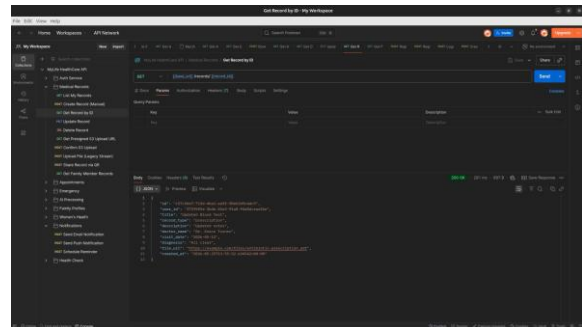
POST/records/



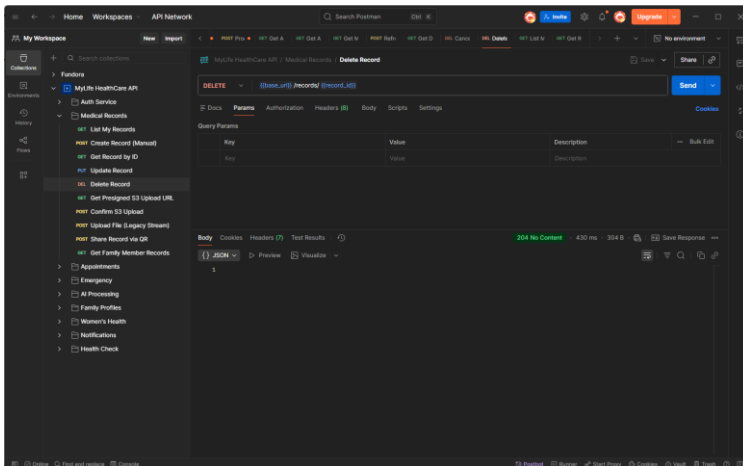
PUT/records/



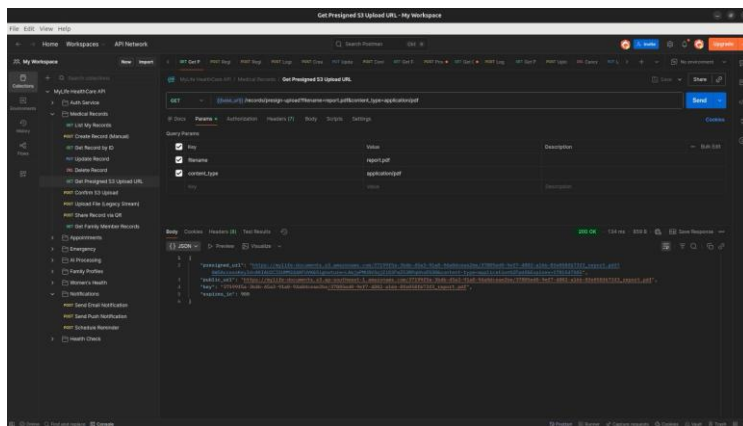
GET/records/{{record_id}}



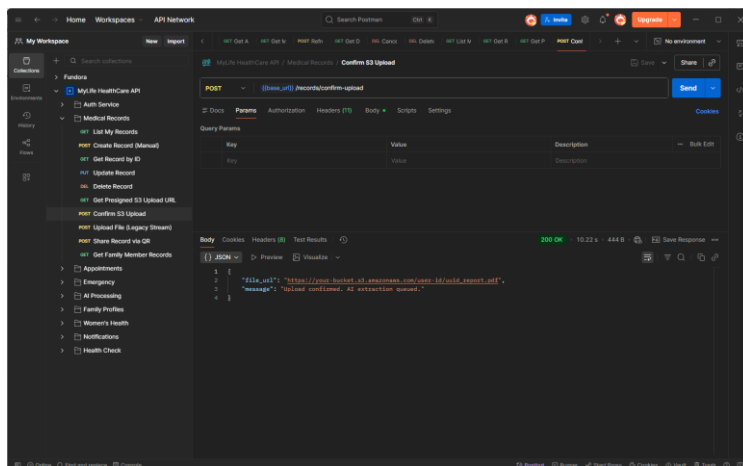
DELETE/records/{ {record_id} }



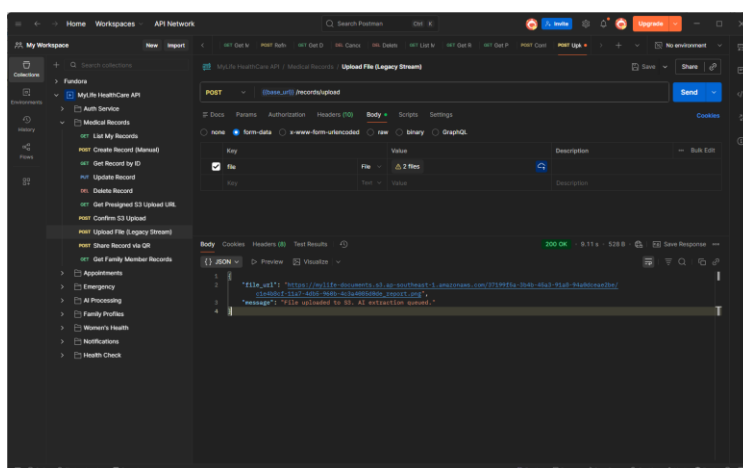
GET/records/presign-upload?filename=report.pdf&content_type=application/pdf



POST/records/confirm-upload

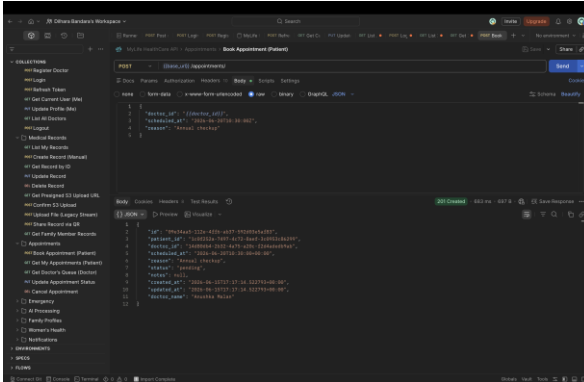


POST/records/upload

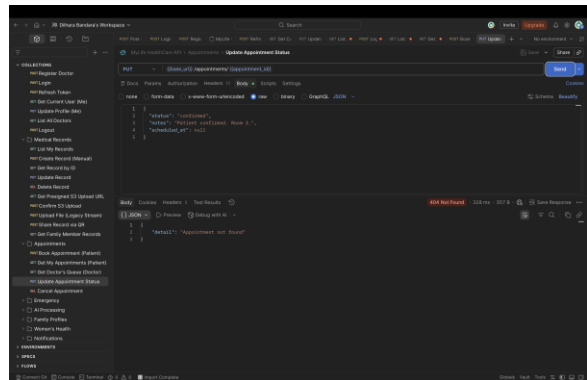


Appointments

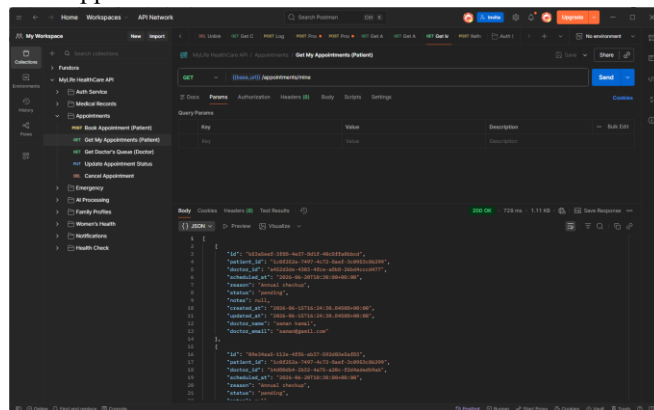
POST/appointment/



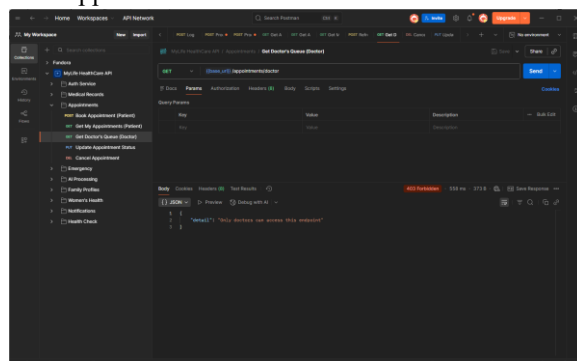
GET/appointment/{{appointment_id}}



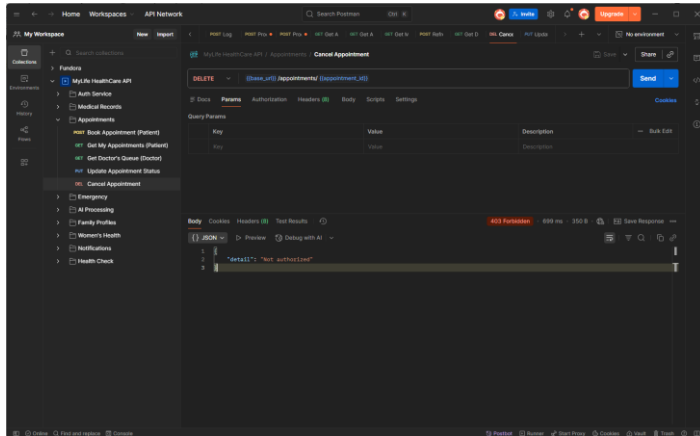
GET/appointments/mine



PUT/appointments/doctor

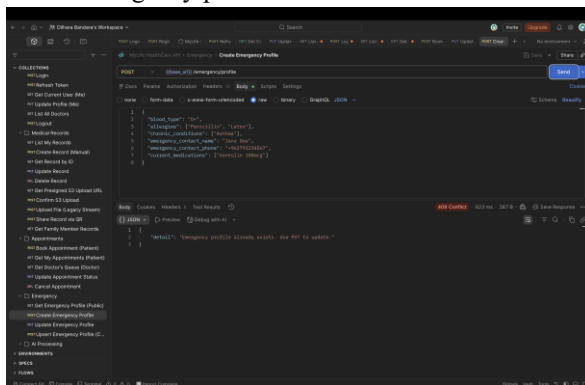


DELETE /appointments/{{appointment_id}}

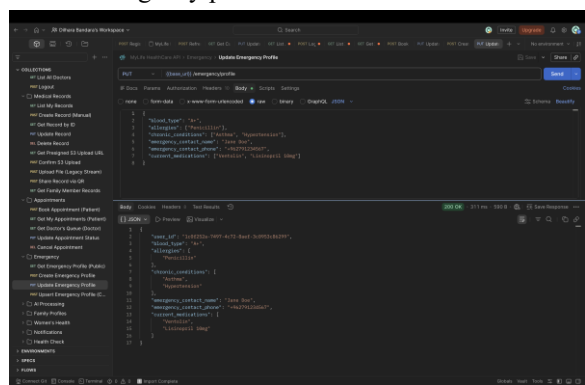


Emergency

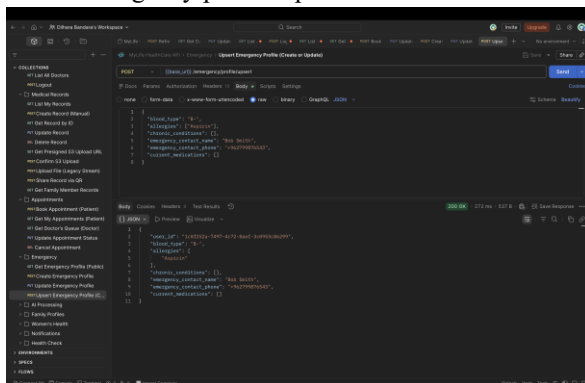
POST/emergency/profile



PUT/emergency/profile



POST/emergency/profile/upsert



3) AI Processing Service

services/ai-processing-service/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import ai_processing
from app.core.config import settings

app = FastAPI(
    title="MYLIFE AI Processing Service",
    description="Extracts structured data from uploaded medical documents using Claude API.",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(ai_processing.router, prefix="/ai", tags=["AI Processing"])

@app.get("/health")
def health_check():
    return {"service": "ai-processing-service", "status": "ok"}
```

services/ai-processing-service/app/routers/ai_processing.py

```
import uuid
import json
import base64
import re
import httpx
import asyncio
import random
import google.generativeai as genai
from google.api_core.exceptions import ResourceExhausted,
GoogleAPICallError
from fastapi import APIRouter, HTTPException, UploadFile, File, Form
from pydantic import BaseModel
from typing import Optional
```

```

from app.core.config import settings
from app.core.database import supabase

router = APIRouter()

genai.configure(api_key=settings.GEMINI_API_KEY)

class ProcessRequest(BaseModel):
    user_id: str
    file_url: str

class AIResultResponse(BaseModel):
    id: str
    user_id: str
    file_url: str
    extracted_data: dict
    confidence_score: Optional[float]
    status: str

EXTRACTION_PROMPT = """
You are an expert medical document analysis AI.
Carefully analyse the provided medical document and extract the following
structured information.

Return ONLY a valid JSON object (no markdown, no commentary) with these
exact keys:
{
    "patient_name": null,
    "patient_dob": null,
    "patient_gender": null,
    "diagnosis": [],
    "medications": [{"name": null, "dosage": null, "frequency": null}],
    "lab_results": [{"test_name": null, "value": null, "unit": null,
"reference_range": null, "status": null}],
    "doctor_name": null,
    "hospital_clinic": null,
    "visit_date": null,
    "allergies": [],
    "follow_up_instructions": null,
    "document_type": null,
    "confidence_score": 0.85
}

Use null for any missing fields. Set confidence_score between 0.0 and 1.0
based on document clarity.

```

```

"""
def _extract_json_from_text(raw_text: str) -> dict:
    try:
        return json.loads(raw_text.strip())
    except json.JSONDecodeError:
        pass
    match = re.search(r"```(?:json)?\s*([\s\S]*?)```", raw_text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r"(\{[\s\S]*\})", raw_text)
    if match:
        try:
            return json.loads(match.group(1))
        except json.JSONDecodeError:
            pass
    return {"raw_response": raw_text, "parse_error": "Could not parse
JSON from Gemini response"}

async def _generate_content_with_retry(model, contents, max_retries=5,
initial_delay=2.0):
    delay = initial_delay
    loop = asyncio.get_running_loop()
    for attempt in range(max_retries):
        try:
            response = await loop.run_in_executor(None, lambda:
model.generate_content(contents))
            return response
        except (ResourceExhausted, Exception) as e:
            err_msg = str(e).lower()
            is_rate_limit = (
                isinstance(e, ResourceExhausted)
                or "429" in err_msg
                or "quota exceeded" in err_msg
                or "resourceexhausted" in err_msg
            )
            if is_rate_limit and attempt < max_retries - 1:
                sleep_time = delay + random.uniform(0, 1.5)
                await asyncio.sleep(sleep_time)
                delay *= 2.0
                continue
            raise e

```

```

async def _call_gemini_with_url(file_url: str) -> tuple[dict, float]:
    model = genai.GenerativeModel("gemini-2.5-flash")

    url_lower = file_url.lower()
    if url_lower.endswith(".pdf"):
        mime_type = "application/pdf"
    elif url_lower.endswith(".png"):
        mime_type = "image/png"
    elif url_lower.endswith(".jpg") or url_lower.endswith(".jpeg"):
        mime_type = "image/jpeg"
    elif url_lower.endswith(".webp"):
        mime_type = "image/webp"
    else:
        mime_type = "image/jpeg"

    try:
        async with httpx.AsyncClient(timeout=30) as http:
            response = await http.get(file_url)
            response.raise_for_status()
            file_bytes = response.content

            gemini_response = await _generate_content_with_retry(model, [
                EXTRACTION_PROMPT,
                {
                    "mime_type": mime_type,
                    "data": base64.b64encode(file_bytes).decode("utf-8"),
                },
            ])
    except Exception:
        gemini_response = await _generate_content_with_retry(model, [
            f"{EXTRACTION_PROMPT}\n\nDocument URL: {file_url}\n"
            "Note: Analyse the document at this URL and extract all
visible medical data."
        ])

    extracted = _extract_json_from_text(gemini_response.text)
    confidence = float(extracted.pop("confidence_score", 0.85))
    return extracted, confidence

async def _store_and_notify(doc_id: str, user_id: str, file_url: str,
                           extracted: dict, confidence: float) -> None:
    supabase.table("extracted_reports").insert({
        "document_id": doc_id,
        "user_id": user_id,
        "extracted_data": extracted,
        "confidence_score": confidence,
    }).execute()

```

```

supabase.table("uploaded_documents") \
    .update({"status": "completed"}) \
    .eq("id", doc_id) \
    .execute()

try:
    async with httpx.AsyncClient() as http:
        await http.post(
            f"{settings.NOTIFICATION_SERVICE_URL}/notify/email",
            json={"user_id": user_id, "event":
"ai_extraction_complete"},
            timeout=5,
        )
except Exception:
    pass

@router.post("/process", response_model=AIResultResponse,
status_code=202)
async def process_document(payload: ProcessRequest):
    doc_id = str(uuid.uuid4())
    supabase.table("uploaded_documents").insert({
        "id": doc_id,
        "user_id": payload.user_id,
        "file_url": payload.file_url,
        "status": "processing",
    }).execute()

    try:
        extracted, confidence = await
_call_gemini_with_url(payload.file_url)
        await _store_and_notify(doc_id, payload.user_id,
payload.file_url, extracted, confidence)

        return AIResultResponse(
            id=doc_id,
            user_id=payload.user_id,
            file_url=payload.file_url,
            extracted_data=extracted,
            confidence_score=confidence,
            status="completed",
        )

    except Exception as e:
        supabase.table("uploaded_documents") \
            .update({"status": "failed"}) \
            .eq("id", doc_id) \
            .execute()

```

```

        raise HTTPException(status_code=500, detail=f"AI extraction
failed: {str(e)}")

@router.post("/process-upload", response_model=AIResultResponse,
status_code=202)
async def process_uploaded_file(
    user_id: str = Form(...),
    file: UploadFile = File(...),
):
    doc_id = str(uuid.uuid4())
    file_bytes = await file.read()
    mime_type = file.content_type or "image/jpeg"

    supabase.table("uploaded_documents").insert({
        "id": doc_id,
        "user_id": user_id,
        "file_url": f"upload://{file.filename}",
        "status": "processing",
    }).execute()

    try:
        model = genai.GenerativeModel("gemini-2.5-flash")
        gemini_response = await _generate_content_with_retry(model, [
            EXTRACTION_PROMPT,
            {
                "mime_type": mime_type,
                "data": base64.b64encode(file_bytes).decode("utf-8"),
            },
        ])
        extracted = _extract_json_from_text(gemini_response.text)
        confidence = float(extracted.pop("confidence_score", 0.85))

        await _store_and_notify(doc_id, user_id,
f"upload://{file.filename}", extracted, confidence)

        return AIResultResponse(
            id=doc_id,
            user_id=user_id,
            file_url=f"upload://{file.filename}",
            extracted_data=extracted,
            confidence_score=confidence,
            status="completed",
        )

    except Exception as e:
        supabase.table("uploaded_documents") \
            .update({"status": "failed"}) \
            .eq("id", doc_id) \

```

```

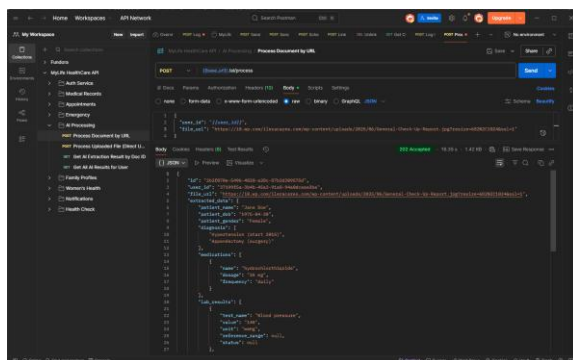
        .execute()
        raise HTTPException(status_code=500, detail=f"AI extraction
failed: {str(e)}")

@router.get("/results/{doc_id}", response_model=AIResultResponse)
async def get_result(doc_id: str):
    result = supabase.table("extracted_reports") \
        .select("*") \
        .eq("document_id", doc_id) \
        .execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Extraction result
not found")
    r = result.data[0]
    return AIResultResponse(
        id=doc_id,
        user_id=r["user_id"],
        file_url="",
        extracted_data=r["extracted_data"],
        confidence_score=r["confidence_score"],
        status="completed",
    )

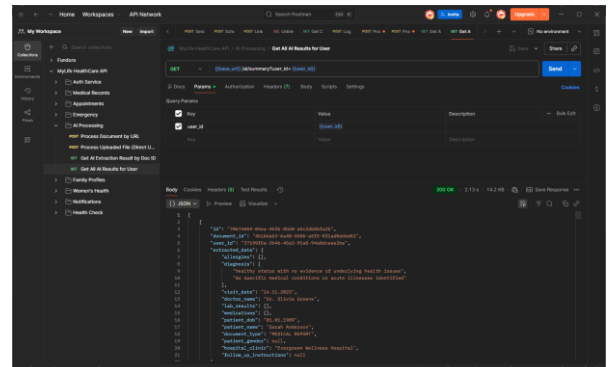
@router.get("/summary")
async def get_summary(user_id: str):
    result = supabase.table("extracted_reports") \
        .select("*") \
        .eq("user_id", user_id) \
        .order("created_at", desc=True) \
        .execute()
    return result.data

```

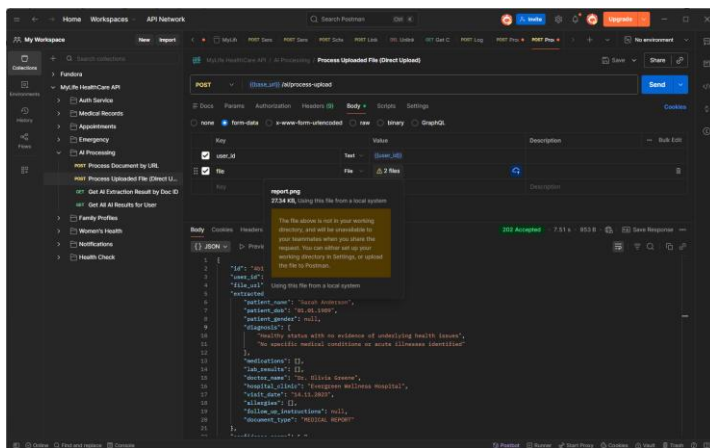
POST/ai/process



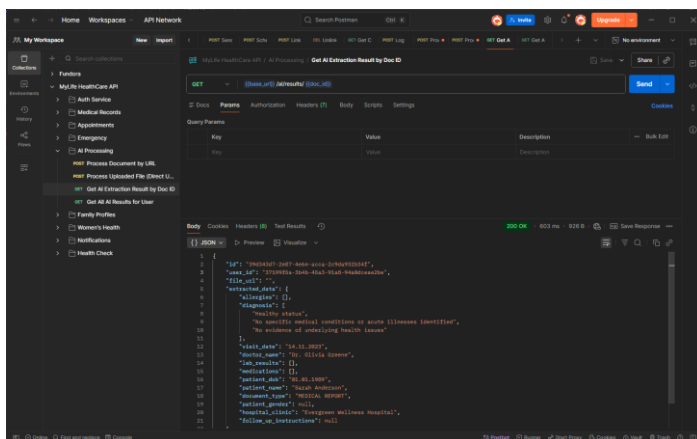
GET/ai/summary?user_id



POST /ai/process-upload



GET /ai/results/{{doc_id}}



4) Family Profiles Service

services/family-profile-service/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import family, health
from app.core.config import settings

app = FastAPI(
    title="MYLIFE Family & Profile Service",
    description="Manages linked family accounts, caregivers, and women's health tracking.",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(family.router, prefix="/family", tags=["Family"])
app.include_router(health.router, prefix="/health", tags=["Women's Health"])

@app.get("/health")
def health_check():
    return {"service": "family-profile-service", "status": "ok"}
```

services/family-profile-service/app/routers/family.py

```
from fastapi import APIRouter, Depends, HTTPException
from app.core.security import get_current_user
from app.core.database import supabase_public, supabase_auth

router = APIRouter()

@router.post("/link")
async def link_family_member(
    linked_user_id: str,
    relationship: str,
    current_user: dict = Depends(get_current_user),
):
    user_check = supabase_auth.table("users") \
```

```

        .select("id, full_name, email, role, gender") \
        .eq("id", linked_user_id) \
        .execute()
    if not user_check.data:
        raise HTTPException(status_code=404, detail="User not found.
Please check the ID.")

    if linked_user_id == current_user["sub"]:
        raise HTTPException(status_code=400, detail="Cannot link yourself
as a family member.")

    existing = supabase_public.table("linked_accounts") \
        .select("id") \
        .eq("owner_id", current_user["sub"]) \
        .eq("linked_user_id", linked_user_id) \
        .execute()
    if existing.data:
        raise HTTPException(status_code=409, detail="This user is already
linked as a family member.")

    result = supabase_public.table("linked_accounts").insert({
        "owner_id": current_user["sub"],
        "linked_user_id": linked_user_id,
        "relationship": relationship,
    }).execute()

    linked_user = user_check.data[0]
    return {
        "message": "Family member linked",
        "data": {
            **result.data[0],
            "full_name": linked_user["full_name"],
            "email": linked_user["email"],
            "role": linked_user["role"],
            "gender": linked_user.get("gender"),
        }
    }

}

@router.get("/members")
async def list_family_members(current_user: dict =
Depends(get_current_user)):
    links = supabase_public.table("linked_accounts") \
        .select("*") \
        .eq("owner_id", current_user["sub"]) \
        .execute()

    if not links.data:
        return []

```

```

enriched = []
for link in links.data:
    user_data = supabase_auth.table("users") \
        .select("id, full_name, email, role, gender") \
        .eq("id", link["linked_user_id"]) \
        .execute()

    user_info = user_data.data[0] if user_data.data else {}
    enriched.append({
        "id": link.get("id"),
        "owner_id": link["owner_id"],
        "linked_user_id": link["linked_user_id"],
        "relationship": link["relationship"],
        "created_at": link.get("created_at"),
        "full_name": user_info.get("full_name", "Unknown"),
        "email": user_info.get("email", ""),
        "role": user_info.get("role", ""),
        "gender": user_info.get("gender"),
    })

return enriched

@router.delete("/unlink/{linked_user_id}")
async def unlink_family_member(linked_user_id: str, current_user: dict =
Depends(get_current_user)):
    supabase_public.table("linked_accounts") \
        .delete() \
        .eq("owner_id", current_user["sub"]) \
        .eq("linked_user_id", linked_user_id) \
        .execute()
    return {"message": "Family member unlinked"}

```

services/family-profile-service/app/routers/health.py

```

from fastapi import APIRouter, Depends
from pydantic import BaseModel
from datetime import date
from typing import Optional
from app.core.security import get_current_user
from app.core.database import supabase

router = APIRouter()

class CycleRequest(BaseModel):

```

```

    start_date: date
    end_date: Optional[date] = None
    cycle_length: Optional[int] = None
    notes: Optional[str] = None

class PregnancyRequest(BaseModel):
    lmp_date: date
    due_date: Optional[date] = None
    notes: Optional[str] = None

@router.post("/cycle")
async def log_cycle(payload: CycleRequest, current_user: dict = Depends(get_current_user)):
    result = supabase.table("menstrual_cycles").insert({
        "user_id": current_user["sub"],
        "start_date": str(payload.start_date),
        "end_date": str(payload.end_date) if payload.end_date else None,
        "cycle_length": payload.cycle_length,
        "notes": payload.notes,
    }).execute()
    return result.data[0]

@router.get("/cycle")
async def get_cycle_history(current_user: dict = Depends(get_current_user)):
    result = supabase.table("menstrual_cycles") \
        .select("*") \
        .eq("user_id", current_user["sub"]) \
        .order("start_date", desc=True) \
        .execute()
    return result.data

@router.post("/pregnancy")
async def log_pregnancy(payload: PregnancyRequest, current_user: dict = Depends(get_current_user)):
    result = supabase.table("pregnancy_records").insert({
        "user_id": current_user["sub"],
        "lmp_date": str(payload.lmp_date),
        "due_date": str(payload.due_date) if payload.due_date else None,
        "notes": payload.notes,
    }).execute()
    return result.data[0]

@router.get("/pregnancy")

```

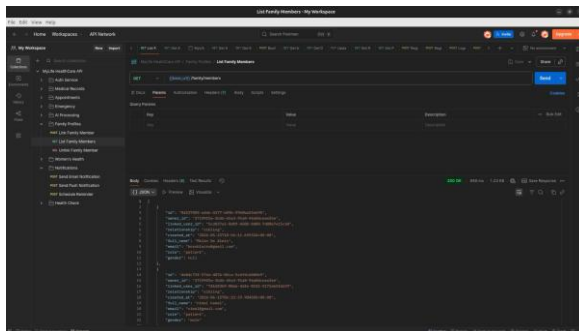
```

async def get_pregnancy_records(current_user: dict =
Depends(get_current_user)):
    result = supabase.table("pregnancy_records") \
        .select("*") \
        .eq("user_id", current_user["sub"]) \
        .execute()
    return result.data

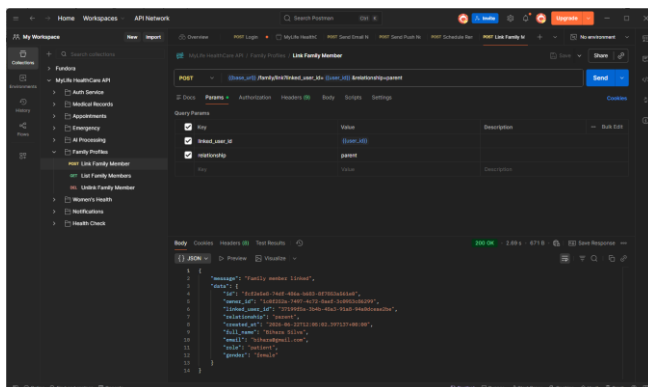
```

Family

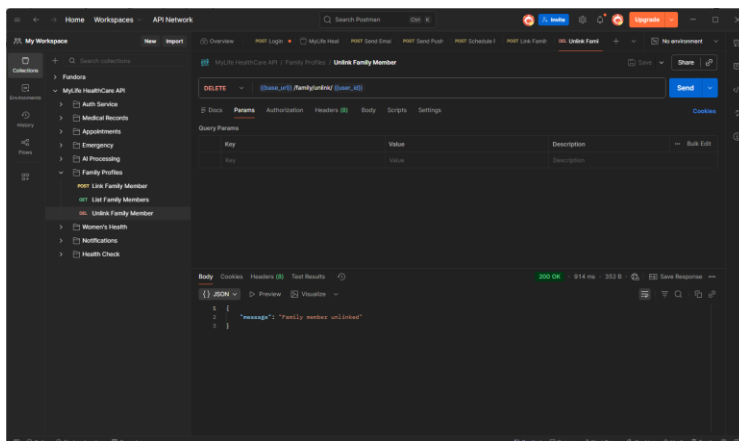
GET/family/members



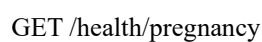
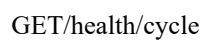
POST /family/link?linked_user_id={{user_id}}&relationship=parent



DELETE/family/unlink/{{user_id}}



POST/health/cycle



5) Notification Service

services/notification-service/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import notifications
from app.core.config import settings

app = FastAPI(
    title="MYLIFE Notification Service",
    description="Sends email and push notifications for key events across the platform.",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(notifications.router, prefix="/notify",
tags=["Notifications"])

@app.get("/health")
def health_check():
    return {"service": "notification-service", "status": "ok"}
```

services/notification-service/app/routers/notifications.py

```
import logging
from fastapi import APIRouter
from pydantic import BaseModel
from typing import Optional
import httpx

from app.core.config import settings
from app.core.database import supabase, supabase_auth

router = APIRouter()
logger = logging.getLogger(__name__)
```



```

_firebase_enabled = False

try:
    import firebase_admin
    from firebase_admin import credentials, messaging

    try:
        firebase_admin.get_app()
        _firebase_enabled = True
    except ValueError:
        try:
            cred =
credentials.Certificate(settings.FIREBASE_CREDENTIALS_PATH)
            firebase_admin.initialize_app(cred)
            _firebase_enabled = True
        except Exception as e:
            logger.warning(f"Firebase init failed (push notifications
disabled): {e}")
    except ImportError:
        logger.warning("firebase-admin not installed - push notifications
disabled")

class EmailRequest(BaseModel):
    user_id: str
    event: str
    record_title: Optional[str] = None

class PushRequest(BaseModel):
    user_id: str
    title: str
    body: str
    fcm_token: Optional[str] = None

class ReminderRequest(BaseModel):
    user_id: str
    reminder_type: str
    scheduled_at: str

EVENT_MESSAGES = {
    "record_created": ("👍 Record Uploaded", "Your medical record
'{title}' has been saved to MYLIFE."),
    "ai_extraction_complete": ("🗉 AI Extraction Done", "Your medical
document has been analysed. View your records."),
    "sos_alert": ("🚑 SOS Alert", "An emergency alert has been triggered
for your account."),

```

```

        "verification": ("🔗 Verify Your Email", "Welcome to MYLIFE! Please
verify your email address."),
    }

async def send_email_via_supabase(to_email: str, subject: str, body:
str):
    try:
        async with httpx.AsyncClient() as client:
            await client.post(
                f"{settings.SUPABASE_URL}/functions/v1/send-email",
                headers={"Authorization": f"Bearer
{settings.SUPABASE_SERVICE_KEY}"},
                json={"to": to_email, "subject": subject, "body": body},
                timeout=10,
            )
    except Exception as e:
        logger.warning(f"Email send failed (non-fatal): {e}")

def send_push_notification(fcm_token: str, title: str, body: str):
    if not _firebase_enabled:
        logger.warning("Push skipped - Firebase not configured")
        return
    try:
        message = messaging.Message(
            notification=messaging.Notification(title=title, body=body),
            token=fcm_token,
        )
        messaging.send(message)
    except Exception as e:
        logger.warning(f"Push send failed (non-fatal): {e}")

@router.post("/email")
async def send_email_notification(payload: EmailRequest):
    user = supabase_auth.table("users").select("email,
full_name").eq("id", payload.user_id).execute()
    if not user.data:
        return {"error": "User not found"}

    email = user.data[0]["email"]
    subject_template, body_template = EVENT_MESSAGES.get(
        payload.event, ("MYLIFE Notification", "You have a new
notification from MYLIFE.")
    )
    body = body_template.replace("{title}", payload.record_title or "")

    await send_email_via_supabase(email, subject_template, body)

```

```

try:
    supabase.table("notification_logs").insert({
        "user_id": payload.user_id,
        "channel": "email",
        "event": payload.event,
        "status": "sent",
    }).execute()
except Exception as e:
    logger.warning(f"Notification log insert failed: {e}")

return {"message": "Email sent"}

@router.post("/push")
async def send_push(payload: PushRequest):
    fcm_token = payload.fcm_token
    if not fcm_token:
        try:
            result =
supabase.table("notifications").select("fcm_token").eq("user_id",
payload.user_id).execute()
            if result.data:
                fcm_token = result.data[0].get("fcm_token")
        except Exception:
            pass

    if fcm_token:
        send_push_notification(fcm_token, payload.title, payload.body)

    try:
        supabase.table("notification_logs").insert({
            "user_id": payload.user_id,
            "channel": "push",
            "event": "manual_push",
            "status": "sent" if (fcm_token and _firebase_enabled) else
"no_token",
        }).execute()
    except Exception as e:
        logger.warning(f"Notification log insert failed: {e}")

    return {"message": "Push notification processed"}

@router.post("/reminder")
async def schedule_reminder(payload: ReminderRequest):
    try:
        supabase.table("notifications").insert({
            "user_id": payload.user_id,

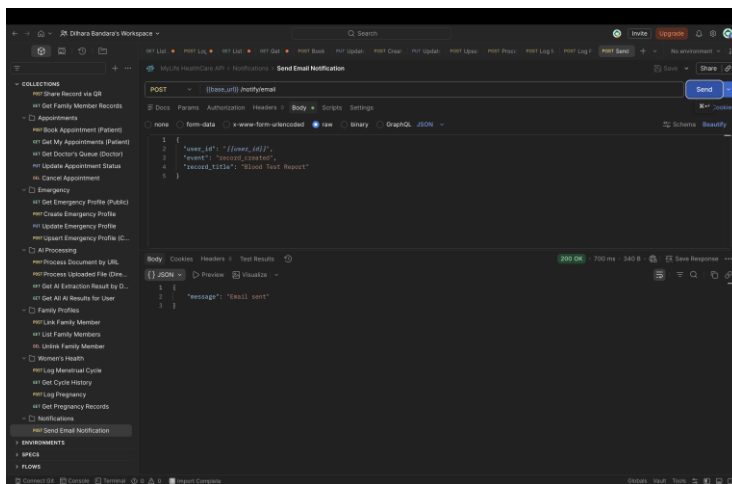
```

```

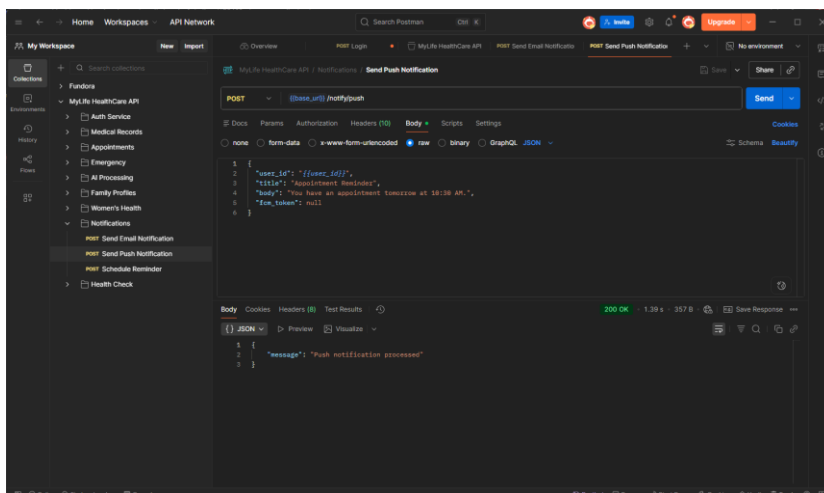
        "reminder_type": payload.reminder_type,
        "scheduled_at": payload.scheduled_at,
        "status": "pending",
    }).execute()
except Exception as e:
    logger.warning(f"Reminder insert failed: {e}")
    return {"message": "Reminder scheduling failed", "detail":
str(e)}
return {"message": "Reminder scheduled"}

```

POST/notify/mail



POST /notify/push



POST /notify/reminder

